

The Crux of Linux Notifier Chains

You can communicate between dynamic modules with notifier chains.

Linux is monolithic like any other kernel. Its subsystems or modules help to keep the kernel light by being flexible enough to load and unload at runtime. In most cases, the kernel modules are interconnected to one another. An event captured by a certain module might be of interest to another module. For instance, when a USB device is plugged to your kernel, the USB core driver has to communicate to the bus driver sitting at the top. This will allow the bus driver to take care of the rest. Another classic example would be of interfaces. Many kernel modules would be looking for a network interface state change. The lower level module that detects the network interface

state change, would communicate this information to the other modules.

Typically, communication systems implement request-reply messaging, or polling. In such models, a program that receives a request will have to send the data available since the last transaction. Such methods sometimes require high bandwidth or they waste polling cycles.

Linux uses a notifier chain, a simple list of functions that is executed when an event occurs. These notifier chains work in a publish-subscribe model. This model is more effective when compared to polling or the request-reply model. In a publish-subscribe model, the 'client' (subscriber) that requires notification of a certain event, 'registers' itself with the 'server' (publisher). The server will